



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет: «Информатика и системы управления» (ИУ)

Кафедра: «Информационная безопасность» (ИУ8)

Домашняя работа № 3

по курсу «Математические основы искусственного
интеллекта»

Выполнил: Земляков А.А.
Группа: ИУ8-13М
Вариант: 3

Проверил: Каганов Ю.Т.

Москва, 2025

1. Цель работы

Изучение алгоритмов многомерного поиска 1-го и 2-го порядка.

Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка.

Вычисление экстремумов функции.

2. Постановка задачи

Требуется найти минимум тестовой функции Розенброка, при $a = 80$, $b = 3$, $f_0 = 110$, $n = 2$.

Были реализованы 4 метода, описанные в задании - метод наискорейшего градиентного спуска, метод Флетчера-Ривза и Полака-Рибьера, метод Девидона-Флэтчера-Паулла и метод Левенберга-Марквардта.

3. Практическая часть

Для реализации данных алгоритмов была написана программа, которая приведена в Листинге А.

Также был построен график функции, для визуализации, он приведен на рисунке 1.

В таблице 1 приведен результат работы программы и сравнение для всех методов, за исключением метода Полака-Рибьера, так как в его случае вызывалось исключение - переполнение памяти.

Функция Розенброка (n=2)
 $a = 80, b = 2, f_0 = 110$

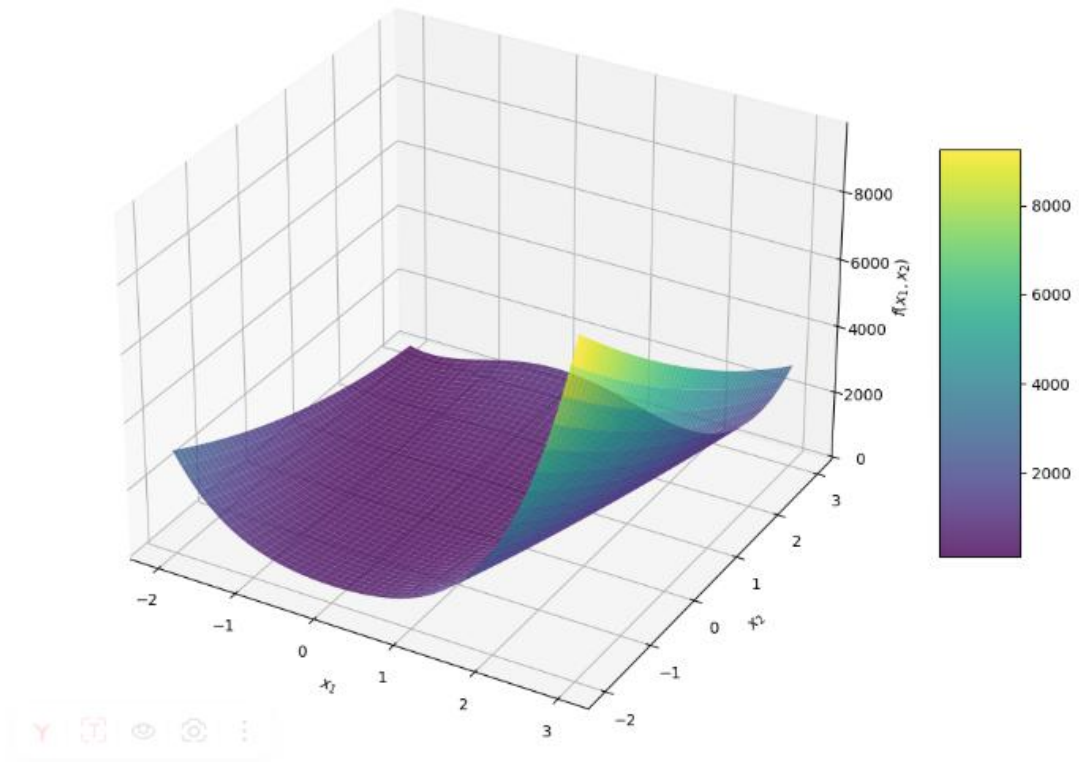


Рисунок 1 - Функция Розенброка

Таблица 1 - Результат работы программы и сравнение методов

Метод	Время, с	Итераций	x	f(x)
Наискорейшего спуска	0.134314	1000	(1.089530, 1.187781)	110.024087
Флетчера-Ривза	0.002608	17	(0.999862, 0.999724)	110
Девидона-Флетчера-Пауэлла	0.001814	10	(1.000000, 1.000000)	110
Левенберга-Макрвардта	0.000814	15	(1.000000, 0.999999)	110

Как видно из результирующей таблицы - самым быстрым методом оказался метод Левенберга-Марквардта, в то же время, он не уступает в точности другим методам, в частности до 9 знаков после запятой полностью совпадают.

Листинг А

```
import time

import numpy as np
from numpy.linalg import inv # инверсия матриц

# Параметры функции
A = 80.0
B = 3.0
F0 = 110.0

# Критерии остановки (пример)
EPS1 = 1e-5 # для нормы градиента
EPS2 = 1e-5 # для разности координат и функции
M = 1000 # максимальное число итераций
MU0 = 1.0 # Для метод Левенберга-Марквардта

def f(x):
    return A * (x[0] ** 2 - x[1]) ** 2 + B * (x[0] - 1) ** 2 + F0

def grad_f(x):
    g1 = -320.0 * x[0] * (x[1] - x[0] ** 2) + 6.0 * (x[0] - 1)
    g2 = 160.0 * (x[1] - x[0] ** 2)
    return np.array([g1, g2])

def hessian_f(x):
    x1, x2 = x[0], x[1]

    H11 = -320.0 * x2 + 960.0 * x1 ** 2 + 6.0
    H12 = -320.0 * x1
    H22 = 160.0

    return np.array([
        [H11, H12],
        [H12, H22]
    ])

def find_alpha_golden_section(x_k, d_k, a_min=0.0, a_max=1.0, tol=1e-8):
    tau = (np.sqrt(5) - 1) / 2
    while f(x_k + a_max * d_k) < f(x_k + a_max * d_k / 2):
```

```

a_max *= 2.0

a, b = a_min, a_max
x1 = a + (1 - tau) * (b - a)
x2 = a + tau * (b - a)
f1 = f(x_k + x1 * d_k)
f2 = f(x_k + x2 * d_k)

while b - a > tol:
    if f1 < f2:
        b = x2
        x2 = x1
        f2 = f1
        x1 = a + (1 - tau) * (b - a)
        f1 = f(x_k + x1 * d_k)
    else:
        a = x1
        x1 = x2
        f1 = f2
        x2 = a + tau * (b - a)
        f2 = f(x_k + x2 * d_k)

return (a + b) / 2

def steepest_descent_method(x0):
    x_k = np.array(x0, dtype=float)
    k = 0
    grad_k = grad_f(x_k)

    history = {'k': [k], 'f': [f(x_k)], 'grad_norm': [np.linalg.norm(grad_k)]}

    if history['grad_norm'][-1] < EPS1:
        print(f'Расчет окончен на k={k}. Точность по градиенту уже достигнута.')
        return x_k, k, 0.0, history

    x_prev = np.copy(x_k)
    f_prev = f(x_k)
    successive_check = 0

    start_time = time.perf_counter()

    while k < M:
        d_k = -grad_k

```

```

alpha_k = find_alpha_golden_section(x_k, d_k)

if alpha_k == 0.0:
    print(f"\nРасчет окончен на k={k}. Линейный поиск не нашел шага (alpha=0).")
    break

x_next = x_k + alpha_k * d_k
grad_next = grad_f(x_next)
f_next = f(x_next)

if np.linalg.norm(grad_next) < EPS1:
    print(f"\nРасчет окончен по критерию градиента ( $\|grad f\| < \{EPS1\}$ ) на k={k + 1}")
    break

condition_x = np.linalg.norm(x_next - x_prev) < EPS2
condition_f = np.abs(f_next - f_prev) < EPS2

if condition_x and condition_f:
    successive_check += 1
else:
    successive_check = 0

if successive_check >= 2:
    print(f"\nРасчет окончен по критерию e2 на k={k + 1}")
    break

x_prev = x_k
f_prev = f_next
x_k = x_next
grad_k = grad_next
k += 1

# Логирование для истории
history['k'].append(k)
history['f'].append(f_next)
history['grad_norm'].append(np.linalg.norm(grad_next))

# Вывод прогресса (опционально)
if k % 1000 == 0:
    print(f"k={k}, f(x)={f_next:.6f}, ||grad||={history['grad_norm'][-1]:.6e}")

end_time = time.perf_counter()
execution_time = end_time - start_time

```

```

if k == M:
    print(f"\nРасчет окончен по критерию максимального числа итераций (M={M}) на k={k}")

return x_k, k, execution_time

def fr_method(x0):
    start_time = time.perf_counter()
    x_k = np.array(x0, dtype=float)
    k = 0

    grad_k = grad_f(x_k)

    # Проверка на начальное условие остановки
    if np.linalg.norm(grad_k) < EPS1:
        print(f"Расчет окончен на k=0. Точность по градиенту уже достигнута.")
        return x_k, k

    d_k = -grad_k

    x_prev = np.copy(x_k)
    f_prev = f(x_k)

    while k < M:
        if np.linalg.norm(grad_k) < EPS1:
            print(f"\nРасчет окончен по критерию градиента ( $\|grad f\| < \{EPS1\}$ ) на k={k}")
            end_time = time.perf_counter()
            execution_time = end_time - start_time
            return x_k, k, execution_time

        alpha_k = find_alpha_golden_section(x_k, d_k)

        x_next = x_k + alpha_k * d_k

        grad_next = grad_f(x_next)

        w_k = np.linalg.norm(grad_next) ** 2 / np.linalg.norm(grad_k) ** 2

        d_next = -grad_next + w_k * d_k

        f_next = f(x_next)

```



```

if (np.linalg.norm(x_next - x_prev) < EPS2) and (np.abs(f_next - f_prev) < EPS2):
    print(f"\nРасчет окончен по критерию  $\epsilon_2$  на  $k=\{k+1\}$ ")
    end_time = time.perf_counter()
    execution_time = end_time - start_time
    return x_next, k + 1, execution_time

# --- Обновление переменных для следующего шага ---
x_prev = x_k
f_prev = f_next
x_k = x_next
grad_k = grad_next
d_k = d_next
k += 1

# Вывод прогресса
if k % 100 == 0:
    print(f"k={k}, f(x)={f_next:.6f}, ||grad||={np.linalg.norm(grad_next):.6e}")

print(f"\nРасчет окончен по критерию максимального числа итераций ( $M=\{M\}$ ) на  $k=\{k\}$ ")
end_time = time.perf_counter()
execution_time = end_time - start_time
return x_k, k, execution_time

def dfp_method(x0):
    x_k = np.array(x0, dtype=float)
    n = len(x_k)
    k = 0
    G_k = np.eye(n)

    grad_k = grad_f(x_k)

    history = {'k': [k], 'f': [f(x_k)], 'grad_norm': [np.linalg.norm(grad_k)]}

    if history['grad_norm'][-1] < EPS1:
        print(f"Расчет окончен на  $k=0$ . Точность по градиенту уже достигнута.")
        return x_k, k, 0.0, history

    x_prev = np.copy(x_k)
    f_prev = f(x_k)
    successive_check = 0

    start_time = time.perf_counter()

```

```

while k < M:
    if np.linalg.norm(grad_k) < EPS1:
        print(f"Расчет окончен по критерию градиента ( $\|grad f\| < \{EPS1\}$ ) на  $k=\{k\}$ ")
        break

    d_k = -G_k @ grad_k

    alpha_k = find_alpha_golden_section(x_k, d_k)

    if alpha_k == 0.0:
        print(f"Расчет окончен на  $k=\{k\}$ . Линейный поиск не нашел шага ( $\alpha=0$ ).")
        break

    x_next = x_k + alpha_k * d_k

    grad_next = grad_f(x_next)

    Delta_g_k = grad_next - grad_k
    Delta_x_k = x_next - x_k

    if np.dot(Delta_x_k, Delta_g_k) <= 1e-10:
        print(f" $k=\{k\}$ : Условие спуска для ДФП не выполнено.  $G^k$  не обновлена.")
        pass
    else:
        term1 = np.outer(Delta_x_k, Delta_x_k) / np.dot(Delta_x_k, Delta_g_k)

        G_grad = G_k @ Delta_g_k
        term2 = np.outer(G_grad, G_grad) / np.dot(Delta_g_k, G_grad)

        G_next = G_k + term1 - term2
        G_k = G_next

    f_next = f(x_next)

    condition_x = np.linalg.norm(x_next - x_prev) < EPS2
    condition_f = np.abs(f_next - f_prev) < EPS2

    if condition_x and condition_f:
        successive_check += 1
    else:
        successive_check = 0

    if successive_check >= 2:
        print(f"Расчет окончен по критерию  $e_2$  на  $k=\{k + 1\}$ ")

```

```

        break

    x_prev = x_k
    f_prev = f_next
    x_k = x_next
    grad_k = grad_next
    k += 1

    history['k'].append(k)
    history['f'].append(f_next)
    history['grad_norm'].append(np.linalg.norm(grad_next))

    if k % 10 == 0:
        print(f'k={k}, f(x)={f_next:.6f}, ||grad||={history['grad_norm'][-1]:.6e}')

    end_time = time.perf_counter()
    execution_time = end_time - start_time

    if k == M:
        print(f'\nРасчет окончен по критерию максимального числа итераций (M={M}) на k={k}')

    return x_k, k, execution_time

def levenberg_marquardt_method(x0):
    x_k = np.array(x0, dtype=float)
    n = len(x_k)
    k = 0
    mu_k = MU0
    I = np.eye(n)
    grad_k = grad_f(x_k)

    history = {'k': [k], 'f': [f(x_k)], 'grad_norm': [np.linalg.norm(grad_k)], 'mu': [mu_k]}

    start_time = time.perf_counter()

    while k < M:
        if np.linalg.norm(grad_k) < EPS1:
            print(f'\nРасчет окончен по критерию градиента (||grad f|| < {EPS1}) на k={k}')
            break
        if k == M:
            print(f'\nРасчет окончен по критерию максимального числа итераций (M={M}) на k={k}')
            break
        H_k = hessian_f(x_k)

```

```
H_augmented = H_k + mu_k * I
```

```
try:
```

```
    H_inv_augmented = inv(H_augmented)
```

```
    d_k = -H_inv_augmented @ grad_k
```

```
except np.linalg.LinAlgError:
```

```
    print(f'k={k}: Матрица H + mu*E вырождена")
```

```
    mu_k = 2 * mu_k
```

```
    continue
```

```
x_next = x_k + d_k
```

```
f_next = f(x_next)
```

```
f_k = f(x_k)
```

```
if f_next < f_k:
```

```
    mu_next = mu_k / 2.0
```

```
    k += 1
```

```
    x_k = x_next
```

```
    grad_k = grad_f(x_k)
```

```
    mu_k = mu_next
```

```
else:
```

```
    mu_next = 2.0 * mu_k
```

```
    mu_k = mu_next
```

```
if f_next < f_k or k == 0:
```

```
    history['k'].append(k)
```

```
    history['f'].append(f(x_k))
```

```
    history['grad_norm'].append(np.linalg.norm(grad_k))
```

```
    history['mu'].append(mu_k)
```

```
if k % 10 == 0 and f_next < f_k:
```

```
    print(f'k={k}, f(x)={f(x_k):.6f}, ||grad||={np.linalg.norm(grad_k):.6e}, mu={mu_k:.6e}")
```

```
end_time = time.perf_counter()
```

```
execution_time = end_time - start_time
```

```
return x_k, k, execution_time
```

```
# Начальная точка
```

```

x_start = [-1.2, 1.0]

print("=== МЕТОД Наискорейшего спуска ===")
x_steepset, k_steepset, execution_time = steepest_descent_method(x_start)
print(f"Общее время выполнения: {execution_time:.6f} секунд")

print("\n" + "=" * 50)
print("      РЕЗУЛЬТАТЫ СРАВНЕНИЯ")
print("=" * 50)
print(f"Теоретический минимум: x* = (1.000, 1.000), f(x*) = {F0}")
print("-" * 50)
print(f"Наискорейшего спуска:")
print(f" Итераций: {k_steepset}")
print(f" Координаты: ({x_steepset[0]:.6f}, {x_steepset[1]:.6f})")
print(f" Значение f(x): {f(x_steepset):.6f}")
print("-" * 50)


print("=== МЕТОД ФЛЕТЧЕРА-РИВЗА (FR) ===")
x_fr, k_fr, execution_time = fr_method(x_start)
print(f"Общее время выполнения: {execution_time:.6f} секунд")

print("\n" + "=" * 50)
print("      РЕЗУЛЬТАТЫ СРАВНЕНИЯ")
print("=" * 50)
print(f"Теоретический минимум: x* = (1.000, 1.000), f(x*) = {F0}")
print("-" * 50)
print(f"Флетчер-Ривз (FR):")
print(f" Итераций: {k_fr}")
print(f" Координаты: ({x_fr[0]:.6f}, {x_fr[1]:.6f})")
print(f" Значение f(x): {f(x_fr):.6f}")
print("-" * 50)


print("=== МЕТОД Девидона-Флетчера-Пауэлла. ===")
x_dfp, k_dfp, execution_time = dfp_method(x_start)
print(f"Общее время выполнения: {execution_time:.6f} секунд")

print("\n" + "=" * 50)
print("      РЕЗУЛЬТАТЫ СРАВНЕНИЯ")
print("=" * 50)
print(f"Теоретический минимум: x* = (1.000, 1.000), f(x*) = {F0}")
print("-" * 50)
print(f"Наискорейшего спуска:")
print(f" Итераций: {k_dfp}")

```

```
print(f" Координаты: ({x_dfp[0]:.6f}, {x_dfp[1]:.6f})")
print(f" Значение f(x): {f(x_dfp):.6f}")
print("-" * 50)

print("=== МЕТОД Левенберга-Макквардта. ===")
x_leven_mar, k_leven_mar, execution_time = levenberg_marquardt_method(x_start)
print(f"Общее время выполнения: {execution_time:.6f} секунд")

print("\n" + "=" * 50)
print("          РЕЗУЛЬТАТЫ СРАВНЕНИЯ")
print("=" * 50)
print(f"Теоретический минимум: x* = (1.000, 1.000), f(x*) = {F0}")
print("-" * 50)
print(f"Наискорейшего спуска:")
print(f" Итераций: {k_leven_mar}")
print(f" Координаты: ({x_leven_mar[0]:.6f}, {x_leven_mar[1]:.6f})")
print(f" Значение f(x): {f(x_leven_mar):.6f}")
print("-" * 50)
```